Web Programming

CSS3

Agenda

- CSS 개요
- CSS 선택자
- CSS 박스모델
- CSS 포지셔닝

CSS개요

CSS 선택자
CSS 박스모델
CSS 포지셔닝

CSS 개요 (1/3)

- 스타일(style)
  - HTML 문서에서 자주 사용하는 글꼴이나 색상, 정렬, 각 요소들의 배치 방법 등 문서의 겉모습을 결정짓는 내용들

- 스타일 시트(style sheet)
  - 스타일을 관리하기 쉽도록 한 군데 모아놓은 것
  - HTML 태그로는 세세한 부분까지 모두 다 지정하기에는 부족
  - 스타일시트를 이용하면 크기, 색상 등의 스타일을 일괄 적용
    - 글자간격, 문단간격, 위치 등 자세한 부분까지 제어

CSS 개요 (2/3)

- 콘텐츠의 내용과 디자인의 분리
  - 마크업 언어 (태그 등): 내용 및 구조에 집중
  - 스타일시트: 외형적 스타일에 집중
  - 내용과 디자인의 분리 – 웹 표준의 시작

- 내용과 디자인 분리의 이점
  - 웹 문서의 내용과 상관없이 디자인만 교체 가능 (CSS Zen Garden)
  - 디자인에 영향 없이 내용만 교체 가능
  - 다양한 기기에 맞춰 탄력적으로 바뀌는 문서 생성 가능
    - 내용은 고정한 채 프린터나 스마트폰 브라우저 등 다양한 기기에 맞는 레이아웃 제작 가능

CSS 개요 (3/3)

- CSS(Cascading Style Sheet)의 특징
  - W3C에서 웹 문서용으로 개발한 스타일시트 언어
  - 기능의 복잡도에 따라 Level1, Level2, Level3로 구분
    - 1996년 CSS Level1 (CSS1), 1998년 CSS Level2 (CSS2)
    - CSS3는 모듈 별로 2005년 이후 개발 중
  - CSS3의 가장 큰 차이점
    - 모듈 기반으로 개발, 디바이스에 따라 원하는 모듈만을 탑재
    - 필요한 모듈만을 빠르게 업데이트 하는 것이 가능
  - CSS3는 화려하고 동적인 스타일 작성 가능
    - 기존의 플래시나 그래픽 디자인 도구에 의존하던 부분을 CSS3 스타일시트 만을 이용하여 상당부분 가능하게 됨

CSS 기본 문법 (1/4)

- 스타일이 적용되지 않은 문서

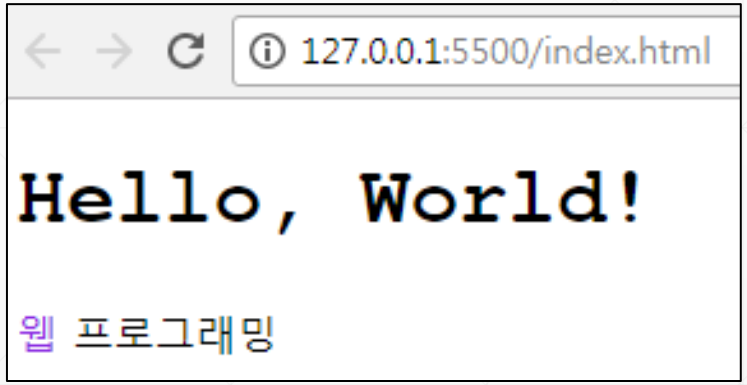
```html
<head>

</head>

<body>
    <h1>Hello, World!</h1>
    <p>웹프로그래밍</p>
</body>
```

CSS 기본 문법 (2/4)

■ 스타일이 적용된 문서

```
<head>
    <style>
        h1 {
            font-family:'Courier New', Courier, monospace;
        }
    </style>
</head>

<body>
    <h1>Hello, World!</h1>
    <p><span style="color: blueviolet">웹 </span>프로그래밍</p>
</body>
```

- H1 태그의 글꼴
- 문자열 중 일부분의 색상

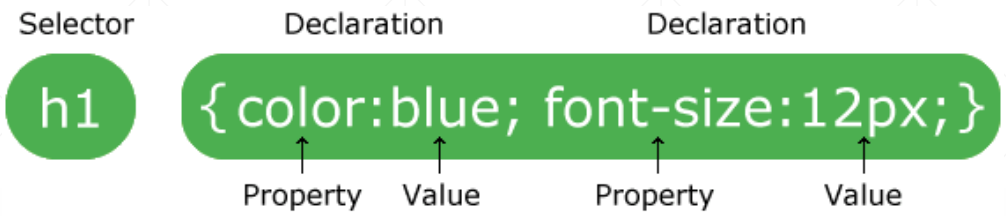


CSS 기본 문법 (3/4)

CSS 기본 문법

- 특정 HTML5 요소 혹은 그 일부분에 대해 CSS 속성(property) 설정

CSS 스타일시트 구성요소



- 선택자 (selector)
  - 스타일을 설정할 대상이 되는 요소
  - 여러 개의 경우 콤마(,)로 구분
- 속성 선언 (property declaration)
  - 속성 (property)과 속성값 (value)
  - 콜론 (:)으로 구분, 세미콜론 (;)으로 종료

CSS 기본 문법 (4/4)

- 실습)
  - 스타일 추가

```
<style>
  h1 {
    font-family: 'Courier New', Courier, monospace;
    color:hotpink;
    font-style:italic;
    text-decoration: underline;
  }
</style>
```

  - 스타일 속성의 이름과 값은 “:” (콜론)으로 구분, “;” (세미콜론)으로 종료
  - 여러 개의 스타일을 적용할 수 있음

CSS 적용 방법 (1/7)

- 내부 스타일 시트 (internal)
  - 웹 문서 내부에 스타일 조절을 위한 CSS 선언
  - <style> 태그 안에 선언
- 인라인 스타일 (inline)
  - HTML 요소에 직접 스타일을 지정하는 방식
  - 해당 요소에 “style” 속성과 CSS 속성/값을 지정하여 스타일 조절
- 외부 스타일 시트 (external)
  - 스타일 시트를 위한 전용 외부 파일 생성
  - 해당 파일의 스타일 설정을 통하여 웹 문서의 스타일 조절
  - 외부 스타일 시트 참조를 위한 링크 필요

CSS 적용 방법 (2/7)

내부 스타일 시트

- <style> 태그에 CSS를 선언하여 스타일 조절
- 문서 내부에 스타일 선언이 존재하는 형태

```
<head>
    <style>
        h1 {
            font-family: 'Courier New', Courier, monospace;
            color: hotpink;
            font-style:italic;
            text-decoration:underline;
        }
    </style>
</head>
```

CSS 적용 방법 (3/7)

인라인 스타일

- 특정 HTML 요소에 CSS를 직접 선언하여 스타일 조절
- 태그의 “style” 속성에 “CSS 속성:CSS 값” 쌍을 추가하여 선언
- 문서 내부에 스타일 선언이 존재하는 형태

```
<body>
    <h1>Hello, World!</h1>
    <p><span style="color: blueviolet"> 웹 </span>프로그래밍</p>
</body>
```

CSS 적용 방법 (4/7)

외부 스타일 시트

- 별도의 CSS 파일에 HTML 요소 스타일 조절을 위한 CSS 선언
- <head>에 <link> 태그를 이용하여 해당 외부 스타일 시트 파일 연결
- 문서 외부에 스타일 선언이 존재하는 형태

- Html 파일
  - <head> 영역에서 아래와 같이 외부 css 파일 링크 설정

```
<link rel="stylesheet" type="text/css" href="css/style.css">
```

css 파일 이름 설정

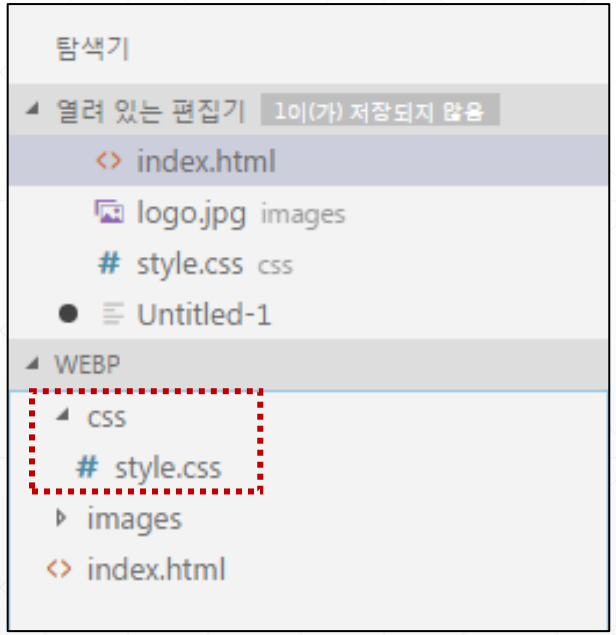
CSS 적용 방법 (5/7)

■ 외부 스타일 시트

- ➢ 내부 스타일 시트와 유사하게 스타일 조절을 위한 CSS 선언
- ➢ 단, <style> 태그는 사용하지 않음

■ 외부 스타일시트 실습)

1. css 서브 폴더 생성
2. css 폴더 밑에 css 파일 생성
3. css 내용 작성
4. html 파일에서 css 파일 연결



CSS 적용 방법 (6/7)

외부 스타일시트 실습)

- html 파일의 내용

```html
<head>
  <title>Web Programming 01</title>
  <link rel="stylesheet" type="text/css" href="css/style.css" />
</head>

<body>
  <h1>Hello, World!</h1>
  <p><span style="color: blueviolet">웹</span>프로그래밍</p>
</body>
```

- css 파일의 내용

```css
h1 {
  font-family: "Courier New", Courier, monospace;
  color: hotpink;
  font-style: italic;
  text-decoration: underline;
}

p {
  color: cyan;
}
```

CSS 적용 방법 (7/7)

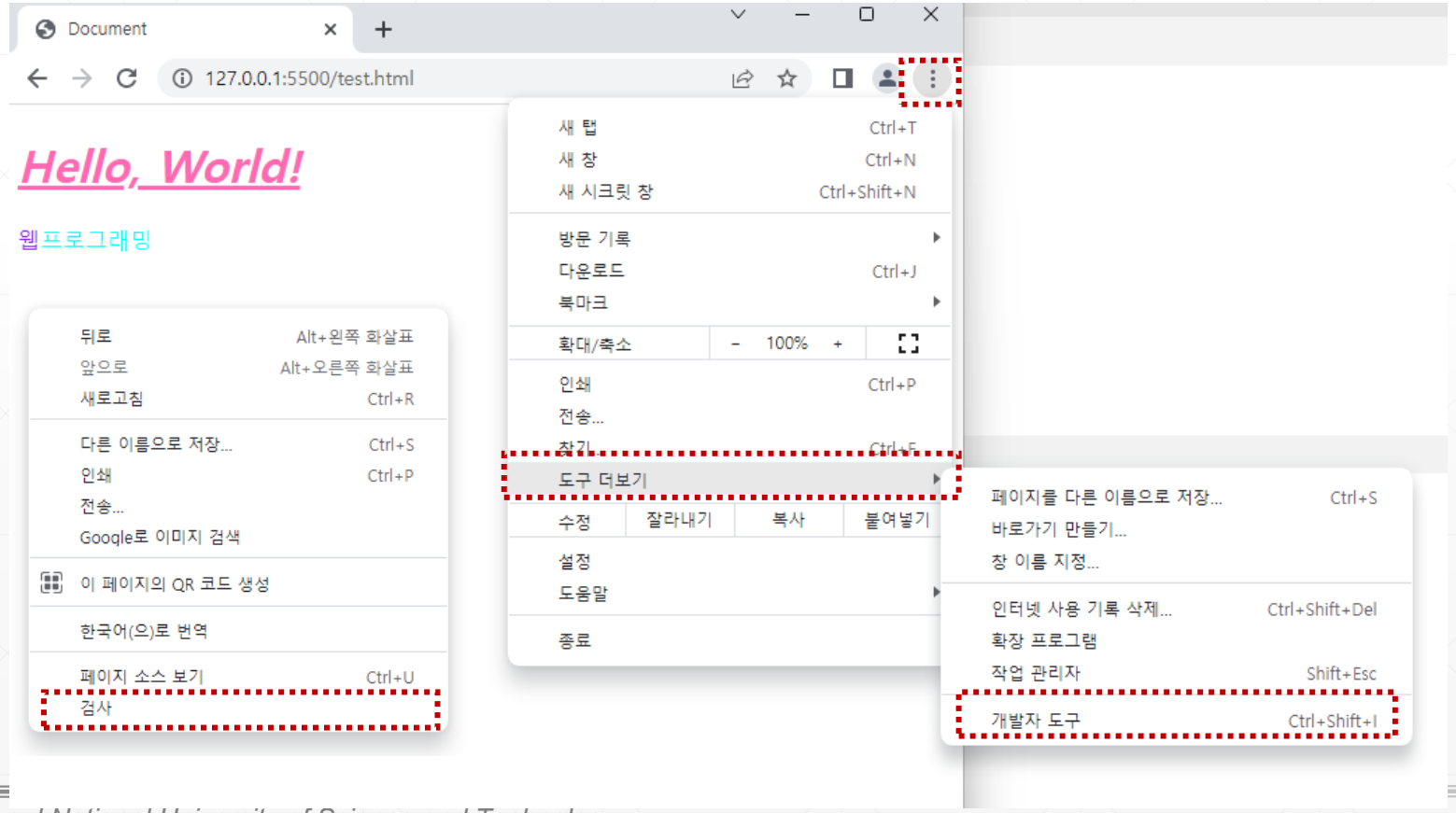
- CSS 우선순위
  - 앞 장의 결과를 확인하였는가? CSS가 잘 동작하는가?
  - “웹 프로그래밍” 문자열의 “웹”은 무슨 색깔인가?

- CSS 적용에는 우선순위가 존재!
  - 중요도, 소스 순서 등에 따라 우선순위가 결정 됨
  - 추후 보다 자세히 다룰 예정

Tips) Chrome Debugger (1/3)

실행 방법

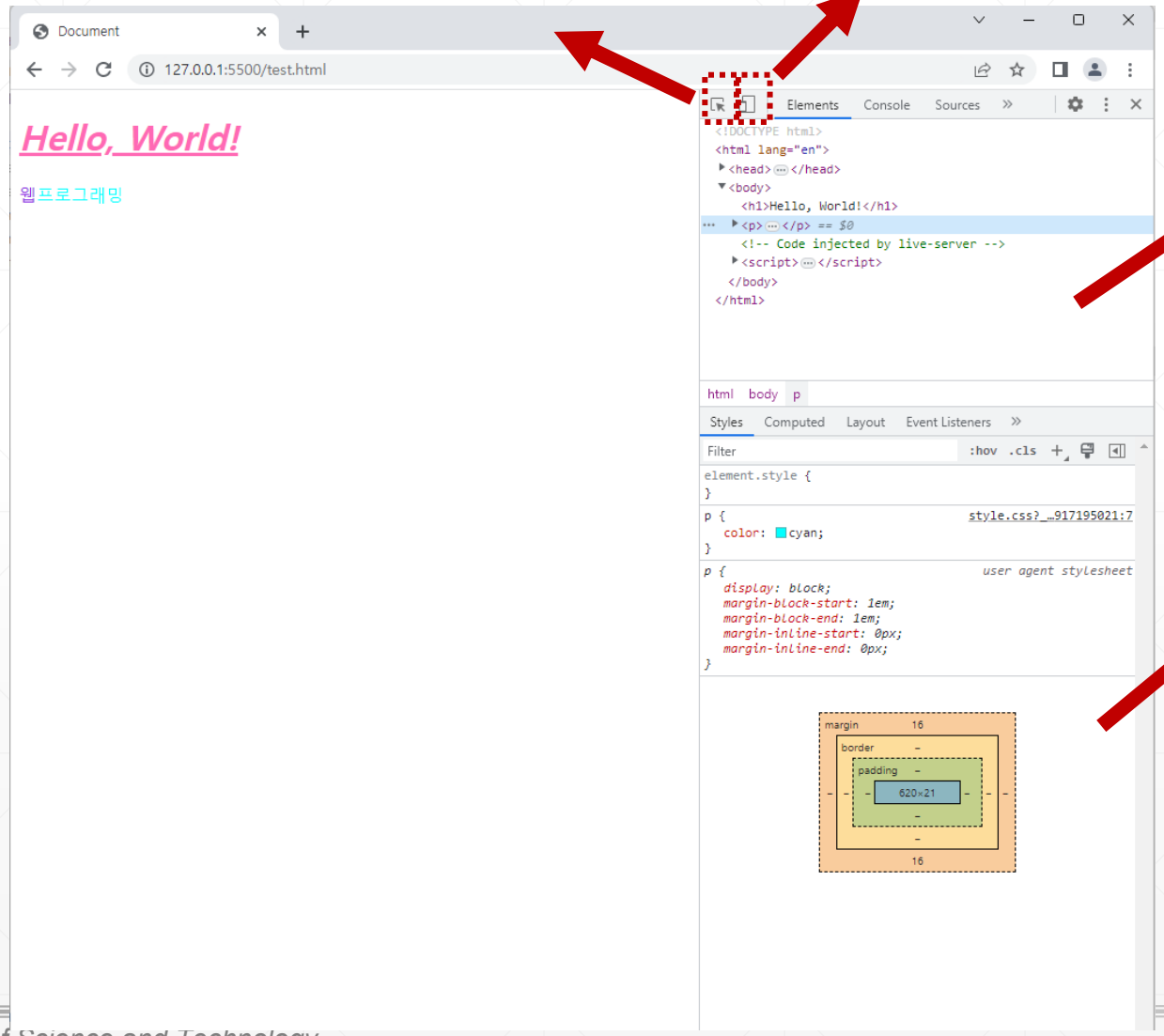
- 크롬 웹페이지 → 우클릭 → 검사(inspect)
- 설정 → 도구 더 보기 → 개발자 도구 (ctrl+shift+I, F12)



Tips) Chrome Debugger (2/3)

Element 선택기능

디바이스 툴바



Element inspector

Style inspector

Tips) Chrome Debugger (3/3)

- Element inspector
  - 웹 문서 요소들에 대한 검사 기능
  - 현재 검사중인 요소는 웹 문서 위에서 하이라이트 됨
  - Element 선택 기능: 검사하고자 하는 요소를 웹 문서에서 선택하면 해당 요소에 대한 내용을 element inspector에서 확인 가능
- Style inspector
  - 선택한 요소에 적용되어 있는 스타일 검사 기능
- 디바이스 툴바
  - 다양한 레이아웃/해상도를 기반으로 웹문서 출력 가능

CSS 개요

CSS 선택자

CSS 박스모델
CSS 포지셔닝

CSS 선택자

- 선택자 (selector)
  - 스타일 설정을 위한 html 요소를 선택하기 위한 도구

```
h1 { color: red; }
```

선택자 스타일 속성 스타일 값

- 전체 선택자
  - 문법

```
* {
    css 선언
}
```

```
선택자 * {
    css 선언
}
```

  - 페이지에 있는 모든 요소를 대상으로 스타일을 적용할 때 사용
  - 특정 선택자의 모든 하위 요소에 스타일을 일괄 적용하려고 할 때

CSS - 전체 선택자 (1/2)

- 전체 선택자 예제)
  - 전체 요소에 적용

```
    <style>
        * {
            font-family: 'Courier New', Courier, monospace;
            font-style: italic;
            color: brown;
        }
    </style>
</head>

<body>
    <h1>Hello, World!</h1>
    <p>
        <span style="color: blueviolet">웹 </span>프로그래밍</p>
</body>
```

CSS - 전체 선택자 (2/2)

전체 선택자 예제)

- 특정 태그의 모든 하위 요소에 적용

```
    <style>
        p * {
            font-family:'Courier New', Courier, monospace;
            font-style: italic;
            color: brown;
        }
    </style>
</head>

<body>
    <h1>Hello, World!</h1>
    <p>
        <span style="color: blueviolet">웹 </span>
        <span>int main()</span>
        프로그래밍
    </p>
</body>
```

<p> 태그의 하위 요소인 태그들에 적용됨

추가!

CSS – 태그 선택자

태그 선택자

- 문서에서 특정 태그를 사용한 모든 요소에 스타일이 적용됨
- 문법

```
태그 {
    css 선언
}
```

- 실습)
  - H1 태그 스타일 지정을 위한 코드 추가

```
h1 {
    color: tomato;
    text-decoration: overline;
}
```

  - HTML 문서에 또 다른 H1 요소도 추가해 보자!

```
<h1>bye, world!</h1>
```

CSS – 클래스 선택자 (1/3)

■ 클래스 선택자

➢ 여러 태그에 특정 스타일을 공통으로 적용하고자 할 때

- class 속성 이용 : <태그이름 class="클래스이름"> … </태그>
- 복수개의 class 할당: <태그이름 class="클래스1 클래스2 클래스3 …">

➢ 클래스 선택자는 클래스 이름 앞에 점(.)을 표기

- ".클래스이름" : 해당 클래스에 모두 적용

  예) .red1 {color: red; font-style: italic; }

  → "red1" 클래스를 가지고 있는 모든 요소들에 적용

- "선택자.클래스이름" : 특정 태그에서 해당 클래스만 지정

  예) strong.red1 {font-size: 12pt }

  → strong 요소 중 "red1" 클래스를 가지고 있는 요소들에 적용

CSS - 클래스 선택자 (2/3)

클래스 선택자 실습

- HTML 문서 (H1 태그에 “intro”, “outro” 클래스 각각 추가)

```html
<h1 class="intro">Hello, World!</h1>
<p>
    <span style="color: blueviolet">웹 </span>
    <span>int main()</span>
    프로그래밍
</p>
<h1 class="outro">Bye, World!</h1>
```

- CSS (H1 태그의 “intro”, “outro” 클래스를 위한 스타일 추가)

```css
h1.intro {
    color: tomato;
    text-decoration: overline;
}

h1.outro {
    color: tan;
    text-decoration: underline;
}
```

CSS – 클래스 선택자 (3/3)

- 클래스 선택자 실습
  - 복수 클래스 사용
  - CSS: “webp” 클래스를 위한 스타일 추가

```
.webp{
  background-color: pink;
}
```

  - HTML: H1 태그에 공통적으로 webp 클래스 추가

```
<h1 class="intro webp">Hello, World!</h1>
<p>
    <span style="color: blueviolet">웹 </span>
    <span>int main()</span>
    프로그래밍
</p>
<h1 class="outro webp">Bye, World!</h1>
```

CSS – ID 선택자 (1/2)

■ ID 선택자

- ➢ 아이디는 html 문서 내에서 (의미적으로) 한 번만 지정
  - <태그이름 id="아이디이름"> … </태그>

- ➢ 해당 아이디로 설정된 태그에만 특정 스타일을 적용
  - 아이디 선택자는 아이디이름 앞에 샵(#)을 표기

    예) #next { color:blue; text-align:center}

    → "next"라는 ID를 가지고 있는 요소에 해당 스타일 적용

CSS – ID 선택자 (2/2)

- ID 선택자 실습)
  - CSS: “mt” id를 위한 스타일 추가

```
#mt {
    text-decoration:line-through;
}
```

  - HTML: “mt” id를 가지는 요소 추가

```
<h1 class="intro webp">Hello, World!</h1>
<p id="mt">
    <span style="color: blueviolet">웹 </span>
    <span>int main()</span>
    프로그래밍
</p>
<h1 class="outro webp">Bye, World!</h1>
```

CSS – 그룹 선택자

- 그룹 선택자
  - 같은 스타일을 사용하는 선택자를 한꺼번에 정의
  - 쉼표(,)로 구분해 여러 선택자를 나열

```
h1 {
    text-align:center; /* 가운데 정렬 */
}
h2 {
    text-align:center; /* 가운데 정렬 */
}
```

→

```
h1, h2 {
    text-align:center; /* 가운데 정렬 */
}
```

CSS – 연결 선택자 (1/4)

연결 선택자 (조합 선택자)

- 하위 선택자
  - 부모 요소에 포함된 **모든 하위 요소**에 스타일 적용
  - 자식 요소뿐만 아니라 손자 요소 (자식의 자식), 손자의 손자 요소 등 모든 하위 요소까지 적용

```
부모요소  하위요소 {
        css 선언
}
```

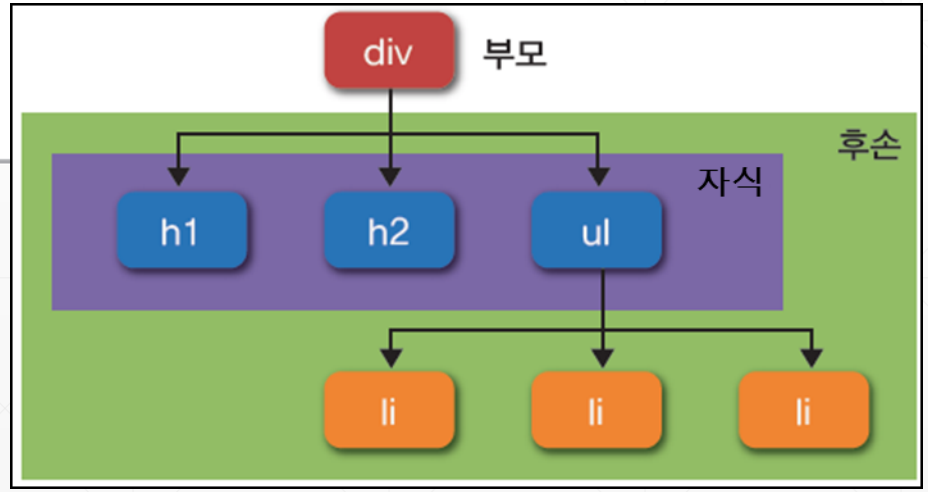
- 자식 선택자
  - **자식 요소**에 스타일을 적용하는 선택자
  - 두 요소 사이에 >(부등호)'를 표시해 부모 요소와 자식 요소를 구분

```
부모요소 > 자식요소 {
        css 선언
}
```

CSS – 연결 선택자 (2/4)

- 부모←→자식/하위(후손) 요소 관계

```
<body>
    <div>
        <h1>CSS3 선택자 기본</h1>
        <h2>기본 선택자</h2>
        <ul>
            <li>전체 선택자</li>
            <li>태그 선택자</li>
            <li>아이디 선택자와 클래스 선택자</li>
        </ul>
    </div>
</body>
```



CSS – 연결 선택자 (3/4)

- 연결 선택자 실습)
  - Html 구성 수정

```
<body>
    <div>
        <h1 class="intro webp">Hello, World!</h1>
        <p id="mt">
          <span style="color: blueviolet">웹</span><span>프로그래밍</span>
        </p>
        <ul>
          <li><p>wow1</p></li>
          <li><p>wow2</p></li>
          <li><p>wow3</p></li>
        </ul>
        <h1 class="outro webp">bye, world!</h1>
    </div>
</body>
```

CSS – 연결 선택자 (4/4)

연결 선택자 실습)

- CSS 구성 (하위 선택자)
  - 의 모든 하위 <p> 요소에 스타일 적용

```
div  p {
    font-weight: bold;
}
```

- CSS 구성 (자식 선택자)
  - 의 자식 <p> 요소에 스타일 적용

```
div > p {
    font-weight: bold;
}
```

CSS – 속성 선택자 (1/2)

- 속성 선택자
  - 지정한 속성을 가진 요소를 찾아 스타일 적용

```
요소[속성] {
        css 선언
}
```

- 속성 값 선택자
  - 주어진 속성과 속성 값이 일치하는 요소를 찾아 스타일 적용

```
요소[속성=값] {
        css 선언
}
```

CSS – 속성 선택자 (2/2)

속성 선택자 실습)

- HTML 구성

```html
<ul>
  <li><p>wow1</p></li>
  <li><img alt="Loading..."></li>
  <li><img alt="N/A"></li>
</ul>
```

- CSS 구성
  - <img> 태그의 ‘alt’ 속성을 보유한 요소에 스타일 적용 (1)
  - <img> 태그의 ‘alt’ 속성 값이 ‘N/A’인 요소에 스타일 적용 (2)

```css
img[alt] {
    color: red;
}
```

(1)

```css
img[alt='N/A'] {
    color: red;
}
```

(2)

CSS – 가상 클래스 선택자

가상 클래스 기반 선택자

- 반응 선택자
  - 사용자의 동작에 반응하는 가상 클래스
  - 예) 링크 관련 상태 (방문/기방문 등), 마우스 위치 상태 등

- 구조 선택자
  - 요소의 구조적인 상태를 한정지을 수 있는 선택자
  - 예) 리스트 요소 중 홀수 번째 요소들만 선택

CSS – 반응 선택자 (1/2)

- 반응 선택자
  - 사용자의 동작에 반응하는 가상 클래스

표기	설명
:link	방문하지 않은 링크에 스타일 적용
:visited	방문한 링크에 스타일 적용
:active	사용자 클릭 등에 의하여 해당 요소가 활성화 된 상태에 대한 스타일 적용
:hover	사용자가 마우스 커서를 올려놓은 상태에 대한 스타일 적용 (link/visited 이후에 선언되어야 정상적으로 동작함)
:focus	입력 요소가 사용자 포커스를 획득한 상태에 대한 스타일 적용
…	…

CSS – 반응 선택자 (2/2)

반응 선택자 예제)

- Html 구성

```html
<a href="#1">Seoultech</a>
<input type="text">
<ul>
  <li><p>wow1</p></li>
  <li><img alt="Loading..."></li>
  <li><img alt="N/A"></li>
</ul>
```

  - <input> 태그는 추후 다룰 예정

- :focus 선택자는 언제 동작하는가?

CSS 구성

```css
a:link {
  color: hotpink;
  text-decoration: none;
}
input:focus {
  background-color: cyan;
}
```

CSS – 구조 선택자 (1/4)

구조 선택자 1

- 순서 중심 구조 선택자

표기	설명
:nth–child(n)	앞에서부터 n번째 자식 요소에 스타일 적용
:nth–last–child(n)	뒤에서부터 n번째 자식 요소에 스타일 적용
:first-child	첫 번째 자식 요소에 스타일 적용
:last-child	마지막 자식 요소에 스타일 적용
…	…

- :nth-child와 :nth-last-child의 경우 an+b와 같은 형식으로 위치를 한정할 수 있음
  - 이 때, n은 0부터 시작

CSS – 구조 선택자 (2/4)

구조 선택자 예제)

- Html 구성

```html
<ul>
  <li><p>wow1</p></li>
  <li><p>wow2</p></li>
  <li><p>wow3</p></li>
  <li><p>wow4</p></li>
  <li><p>wow5</p></li>
</ul>
```

- CSS 구성
  - <ul> 요소의 첫 번째 자식 요소인 <li> 요소에 스타일 적용

```css
ul li:first-child{ text-decoration: line-through red double;  }
ul li:nth-child(1){ text-decoration: line-through red double; }
```

  - <ul> 요소의 짝수 번째 자식 요소인 <li> 요소에 스타일 적용

```css
ul li:nth-child(2n){ text-decoration: line-through red double; }
ul li:nth-child(even){ text-decoration: line-through red double; }
```

CSS – 구조 선택자 (3/4)

구조 선택자 2

- 요소 중심 구조 선택자

표기	설명
:nth–of–type(n)	앞에서부터 n번째 해당 타입의 자식 요소에 스타일 적용
:nth–last–of–type(n)	뒤에서부터 n번째 해당 타입의 자식 요소에 스타일 적용
:first-of-type	첫 번째 해당 타입의 자식 요소에 스타일 적용
:last-of-type	마지막 해당 타입의 자식 요소에 스타일 적용
…	…

- :nth-of-type과 :nth-last-of-type의 경우 an+b와 같은 형식으로 위치를 한정할 수 있음
  - 이 때, n은 0부터 시작

CSS – 구조 선택자 (4/4)

구조 선택자 예제)

- Html 구성

```html
<ul>
    <p>Test</p>
    <li><p>wow1</p></li>
    <li><p>wow2</p></li>
    <li><p>wow3</p></li>
    <li><p>wow4</p></li>
    <li><p>wow5</p></li>
</ul>
```

- CSS 구성
  - <ul> 요소의 첫 번째 <li> 요소에 스타일 적용

```css
ul li:nth-of-type(1){ text-decoration: line-through red double; }
ul li:first-of-type{ text-decoration: line-through red double; }
```

  - <ul> 요소의 짝수 번째 <li> 요소에 스타일 적용

```css
ul li:nth-of-type(2n){ text-decoration: line-through red double; }
ul li:nth-of-type(even){ text-decoration: line-through red double; }
```

CSS – 선택자 우선순위 (1/9)

- 동일 요소에 CSS 스타일 선언이 충돌한다면?
- CSS 스타일 적용의 우선순위
  1. 인라인 스타일
  2. ID 선택자에 의한 스타일
  3. 클래스, 속성, 가상 클래스 선택자에 의한 스타일
  4. 태그에 의한 스타일
  - 우선순위가 동일할 경우 나중에 선언된 스타일이 앞서 선언된 스타일을 덮어 쓰게 됨
- 예외) 중요 스타일 선언 (!important)
  - 스타일 선언 뒤에 “!important” 키워드 할당 시 최우선순위 부여
  - Bad practice

CSS – 선택자 우선순위 (2/9)

- CSS 우선순위 예제)

```
<div>
        <h1 class="intro webp">Hello, World!</h1>
        <p id="mt">
                <span style="color:blueviolet"> 웹 </span>
                <span> int main()</span>
        프로그래밍
        </p>

        <h1 class="outro">Bye, World!</h1>
</div>
```

CSS – 선택자 우선순위 (3/9)

- CSS 우선순위 예제)

```
p * {
font-family: 'Courier New',
Courier, monospace;
font-style: italic;
color: brown;
}

h1.intro {
color: tomato;
text-decoration: overline;
}

h1.outro {
color: tan;
text-decoration: underline;
}
```

```
.webp {
background-color: snow;
}

#mt {
text-decoration: line-through;
}
```

CSS – 선택자 우선순위 (4/9)

CSS 스타일 적용의 우선순위
1. 인라인 스타일
2. ID 선택자에 의한 스타일
3. 클래스, 속성, 가상 클래스 선택자에 의한 스타일
4. 태그에 의한 스타일

Inline: 최상위 우선순위

- <p> 태그의 하위 요소들이 모두 동일한 색상을 가지는가?

```
p * {
    font-family: 'Courier New', Courier, monospace;
    font-style: italic;
    color: brown;
}
```

- "웹"이라는 글씨는 보라색! (inline style이 우선됨)

```
<p id="mt">
    <span style="color: blueviolet"> 웹 </span>
    <span> int main()</span>
    프로그래밍
</p>
```

CSS – 선택자 우선순위 (5/9)

CSS 스타일 적용의 우선순위
1. 인라인 스타일
2. ID 선택자에 의한 스타일
3. 클래스, 속성, 가상 클래스 선택자에 의한 스타일
4. 태그에 의한 스타일

- ID: 클래스/속성/태그 선택자에 앞서는 우선순위
  - 일반적인 <p> 태그를 위한 스타일을 추가
  - #mt 를 위한 스타일 추가

```
#mt {
    text-decoration: line-through;
    background-color: cyan;
}

p { background-color: black;}
```

  - <p> 태그에 webp 클래스 추가 (#ID의 배경 스타일이 우선 적용됨)

```
<p id="mt" class='webp'>
    <span style="color: blueviolet"> 웹 </span>
    <span> int main()</span>
    프로그래밍
</p>
```

ID에 의한 스타일 적용이 우선됨!

CSS – 선택자 우선순위 (6/9)

CSS 스타일 적용의 우선순위
1. 인라인 스타일
2. ID 선택자에 의한 스타일
3. 클래스, 속성, 가상 클래스 선택자에 의한 스타일
4. 태그에 의한 스타일

- 클래스/속성/가상클래스: 태그에 앞서는 우선순위
  - 일반적인 <h1> 태그를 위한 스타일 추가

```
h1 {color: red;}
```

  - <h1 class='outro'>와 color 스타일에 대한 충돌

```
h1.outro {
    color: tan;
    text-decoration: underline;
}
```

  - 클래스 선택자가 태그 선택자에 우선함!

CSS – 선택자 우선순위 (7/9)

- 또다른 우선순위 충돌 예
  - H1.intro와 .webp 스타일을 충돌시켜 보자!

```
h1.intro {
    color: tomato;
    text-decoration: overline;
}

h1.outro {
    color: tan;
    text-decoration: underline;
}

.webp {
    background-color: snow;
    text-decoration: none;
}
```

CSS – 선택자 우선순위 (8/9)

동일 우선순위 간의 충돌

- .webp2 클래스에 대한 스타일을 맨 마지막에 추가

⋮

```
    .webp2 {background-color: hotpink; }
</style>
```

- <h1> 태그에 webp 클래스와 webp2 클래스를 동시에 부여!
  - 동일 우선순위 선택자들간에 background-color에 대한 충돌이 발생

```
<h1 class="outro webp2 webp ">Bye, World!</h1>
```

- 나중에 선언된 스타일이 덮어쓰는 구조!

CSS – 선택자 우선순위 (9/9)

■ 스타일 강제 적용

- !important 키워드: 우선순위에 상관없이 강제 스타일 적용 (aka 하.지.마.)

- 스타일 최상단 위치에 아래 스타일 추가
  - 일부 텍스트를 제외하고는 아래 스타일은 적용되지 않음

```
<style>
    * { color: white;}
    p * {
        font-family: 'Courier New', Courier, monospace;
        font-style: italic;
        color: brown;
    }
```

  - 만약 !important 키워드를 추가한다면?

```
* { color: white !important;}
```

CSS – 기타 선택자

- 그 외 선택자
  - CSS selector reference 참고
  - Basic CSS selectors - Learn web development | MDN

- CSS 선택자 데모
  - https://www.w3schools.com/cssref/trysel.asp
  - 각 선택자의 동작 방식에 대하여 확인할 수 있음

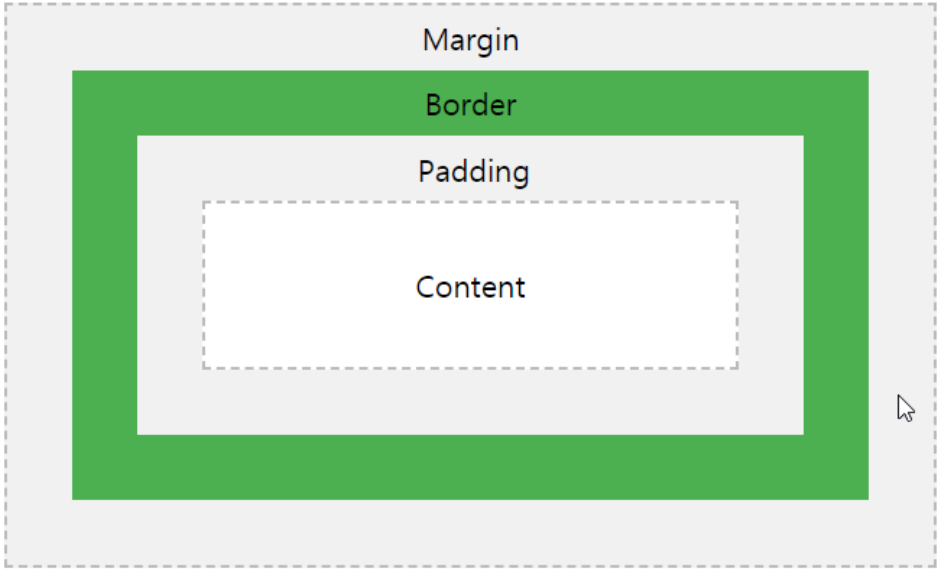
CSS개요
CSS 선택자

CSS 박스모델

CSS 포지셔닝

박스 모델 (1/6)

- 박스 (Box) 모델
  - 모든 HTML 요소들은 아래와 같이 자리를 차지하는 박스로 간주됨
  - Content: HTML 요소의 내용
  - Padding: 테두리와 내용 사이의 내부 여백
  - Border: 경계선
  - Margin: 경계선 외부의 여백



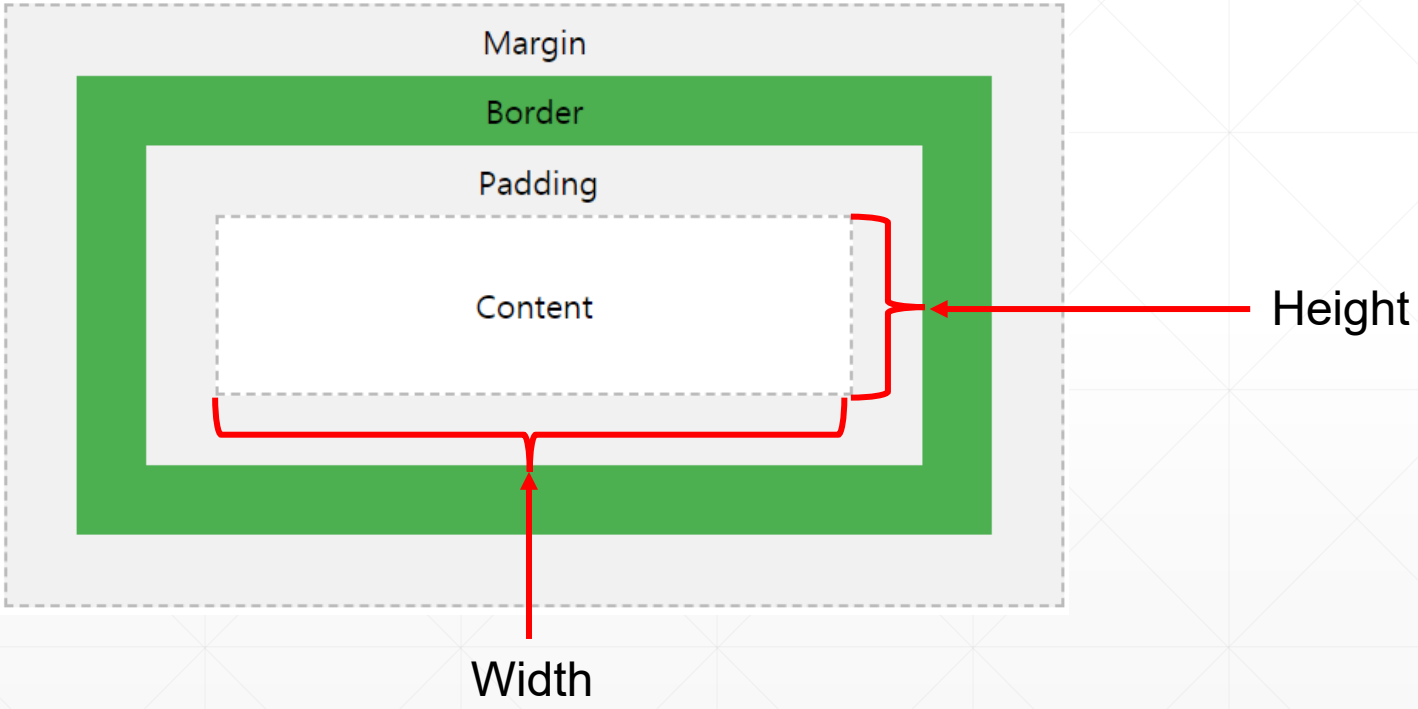
박스 모델 (2/6)

- 요소(element)의 가로 및 세로 길이
  - Width 속성: HTML5 요소의 가로 길이 조절
  - Height 속성: HTML5 요소의 세로 길이 조절

- 길이의 단위:
  - 절대 단위: cm(센치미터), px(픽셀) 등
  - 상대 단위
    - %(백분율): 부모 컨테이너의 가로 길이 * 백분율에 비례하게 설정
    - em: 폰트 크기에 비례하게 설정 (1em = 1배수, 1.5em = 1.5배수)
    - …

박스 모델 (3/6)

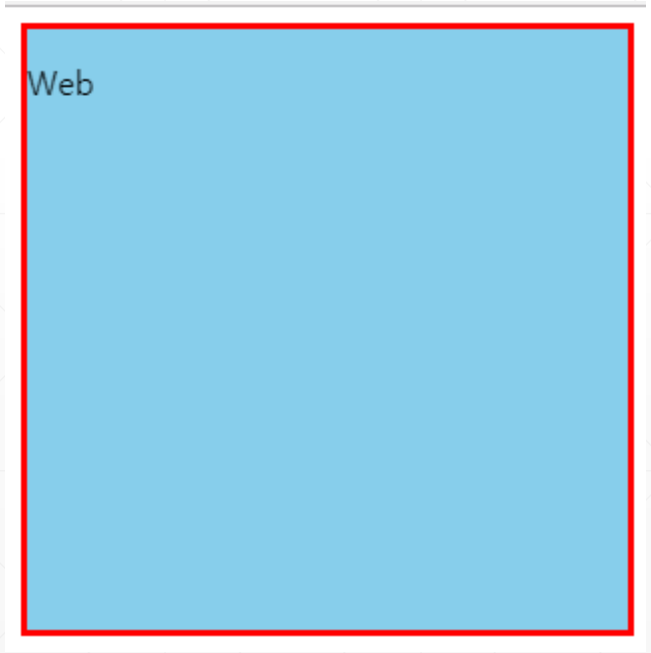
- 가로(width)/세로(height) 속성



박스 모델 (4/6)

- 박스 모델 예제)

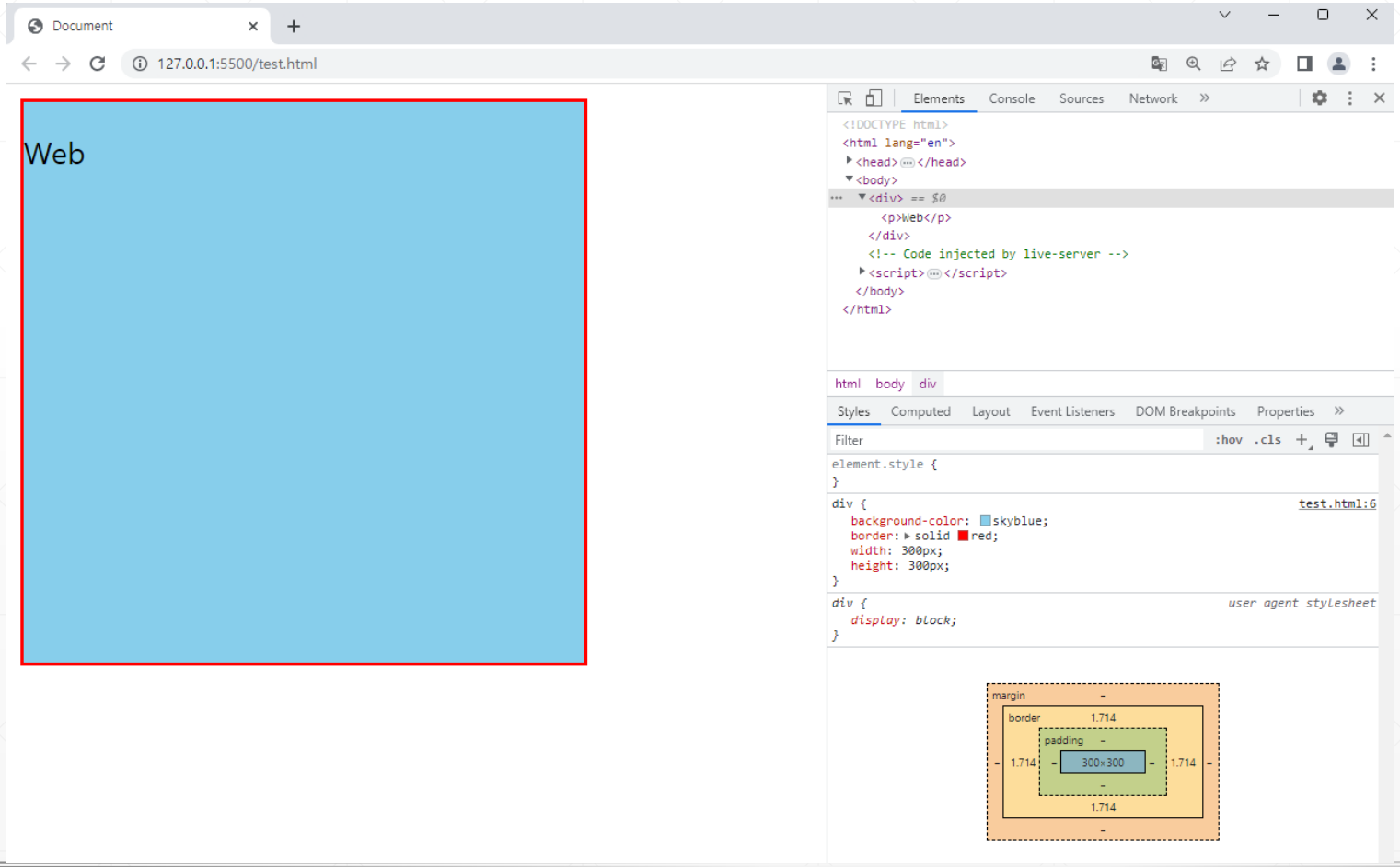
```
<head>
<style>
    div {
        background-color: skyblue;
        border: solid red;
        width: 300px;
        height: 300px;
    }
</style>
</head>

<body>
    <div>
        <p>Web</p>
    </div>
</body>
```



박스 모델 (5/6)

박스 모델 예제)

- 크롬 개발자 도구(F12)를 통한 요소 및 스타일 검사



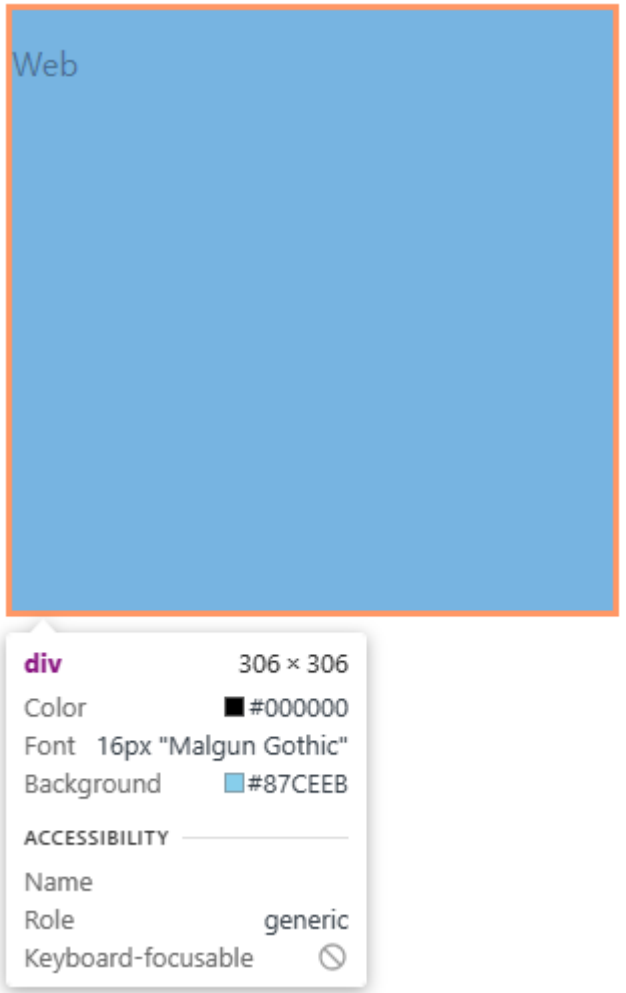
박스 모델 (6/6)

박스 모델 예제)

- 전체 요소의 가로 및 세로 길이 확인
  - 가로: 좌/우 margin + 좌/우 경계선 폭 + 좌/우 padding + 내용 가로 길이
  - 세로: 상/하 margin + 상/하 경계선 폭 + 상/하 padding + 내용 세로 길이
  - 인스펙터는 margin을 제외한 영역 길이를 계산해서 보여줌
    - 경계선 폭 + padding + 가로/세로 길이

- <p> 요소의 가로 및 세로 길이는?

- 개발자 도구에서 가로/세로 길이 수정해보기
  - 요소의 가로길이를 %, px, em단위로 수정
  - Ex) 100%, 100px, 1em

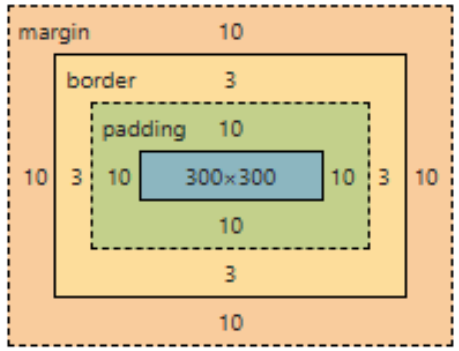


박스 모델: Margin & Padding (1/3)

- Margin 및 padding의 조절
  - 절대단위(px/cm)와 상대단위(%,em)를 이용하여 조절 가능
  - 인스펙터에서도 조절해보자!

```
<style>
    div {
        background-color: skyblue;
        border: solid red;
        width: 300px;
        height: 300px;
        margin: 10px;
        padding: 10px;
    }
</style>
```

```
div {
   background-color: skyblue;
   border: ▶ solid red;
   width: 300px;
   height: 300px;
   margin: ▶ 10px;
   padding: ▶ 10px;
}
div {                                  사용자 에이전트
   display: block;
}
```



박스 모델: Margin & Padding (2/3)

- Margin 및 padding의 조절

Margin 속성	**Padding 속성**	의미
margin	padding	margin/padding 일괄 조절
margin-top	padding-top	상단 margin/padding 조절
margin-right	padding-right	우측 margin/padding 조절
margin-bottom	padding-bottom	하단 margin/padding 조절
margin-left	padding-left	좌측 margin/padding 조절

- margin 속성의 또다른 입력 패턴
  - margin: top right bottom left; (입력 인자가 4개)
  - margin: top left/right bottom; (입력 인자가 3개)
  - margin: top/bottom left/right; (입력 인자가 2개)

박스 모델: Margin & Padding (3/3)

- margin 입력 예제)
- margin-collapse?
  - 두 요소의 상하 margin이 겹칠 경우, 큰 쪽 기준으로 병합됨
  - .box1의 하단과 .box2의 상단
  - .box2의 하단과 .box3의 상단
  - 좌/우는 병합되지 않음!

```
…

.box1 { margin: 30px 10px 30px;}
.box2 { margin: 50px 50px;}
.box3 { margin: 10px 20px 30px 40px;}

</style>
</head>

<body>
    <div>box</div>
    <div class=box1>box1</div>
    <div class=box2>box2</div>
    <div class=box3>box3</div>
</body>
```

박스 모델: Display (1/6)

HTML 요소의 디스플레이 방식

- 블록 레벨 (block level)
  - 언제나 새로운 단락(newline)에서 시작함
  - 가로는 가능한 최대 크기를 차지함
  - 또다른 블록 레벨 요소나 인라인 요소들을 포함할 수 있음

<address>	<dd>	<figcaption>	<header>	<noscript>	<section>
<article>		<figure>	<hr>	<ol>	<table>
<aside>	<dl>	<footer>	<li>	<output>	<tfoot>
<blockquote>	<dt>	<form>	<main>	<p>	<ul>
<canvas>	<fieldset>	<h1>-<h6>	<nav>	<pre>	<video>

- 인라인 (inline)
  - 새로운 단락에서 시작하지 않음 (한 행에 연속해서 붙이려고 함!)
  - 필요한 만큼만의 가로 크기를 차지함
  - 또다른 인라인 요소들을 포함할 수 있음

<a>	<big>	<dfn>	<kbd>	<samp>		<tt>
<abbr>	 		<label>	<script>	<sub>	<var>
<acronym>	<button>	<i>	<map>	<select>	<sup>	
<b>	<cite>	<img>	<object>	<small>	<textarea>	
<bdo>	<code>	<input>	<q>		<time>	

박스 모델: Display (2/6)

- 블록 레벨 요소들의 디스플레이 방식

```html
<body>
  <div>box1</div>
  <div>box2</div>
  <div>box3</div>
  <div>box4</div>
</body>
```

Html 구조

```css
<style>
  div {
    background-color: skyblue;
    margin: 10px;
  }
</style>
```

CSS 구조

- 는 대표적인 블록 레벨 요소
- 는 위에서 아래로 쌓이는 형태로 공간을 차지함
- 실행 결과)

box1
box2
box3
box4

박스 모델: Display (3/6)

인라인 요소들의 디스플레이 방식

Html 구조 `<span>span1</span><span>span2</span><span>span3</span><span>span4</span>`

CSS 구조 `span{border:solid;margin: 10px;}`

- 은 대표적인 인라인 요소
- 은 좌에서 우로 진행되는 형태로 공간을 차지함
- 인라인 요소들에는 height/width 및 상하단 margin 설정 불가
- 실행 결과)

box1
box2
box3
box4
Span1 Span2 Span3 Span4

박스 모델: Display (4/6)

- Display 속성: 디스플레이 방식을 변경하기 위한 속성
  - inline: inline 방식으로 설정
  - block: block 방식으로 설정
  - inline-block: 내용은 block으로 배치는 inline 방식으로 설정
  - 예제) 와 의 display 방식을 각각 inline, block으로 변경

```
div {
    display:inline;
    background-color: skyblue;
    margin:10px;
}
span {
    display:block;
    border:solid;
    margin:10px;}
```

box1 box2 box3 box4

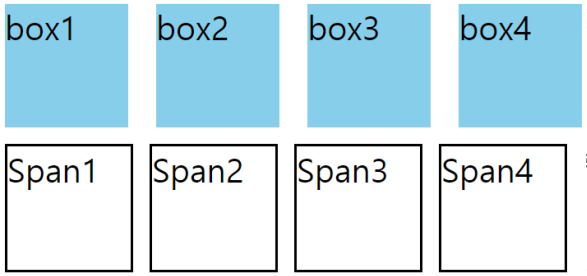
Span1

Span2

Span3

Span4

박스 모델: Display (5/6)



- display: inline-block 설정
  - box를 inline 형태로 배치하기 위한 설정
  - 내용은 block-level, 배치는 inline 형태
    - 따라서, 가로/세로 길이 설정 및 상/하단 margin 설정이 가능함

```
div {
    display:inline-block;
    background-color: skyblue;
    width:150px;
    height:150px;
    margin:10px;
}
```

```
span {
    display:inline-block;
    border:solid;
    width:150px;
    height:150px;
    margin:10px;
}
```

박스 모델: Display (6/6)

- display: none
  - 배치 삭제를 위한 설정

- display 속성 정리
  - inline: 해당 요소를 inline 형태로 출력
  - block : 해당 요소를 block 형태로 출력
  - inline-block : 해당 요소를 inline-block 형태로 출력
  - none : 해당 요소를 화면에 표시하지 않음 (자리를 차지하지 않음)
  - 기타 (레퍼런스 참조)

CSS개요
CSS선택자
CSS 박스모델

CSS 포지셔닝

CSS 포지셔닝

- HTML 요소들을 페이지에 적절히 배치하기 위한 설정
  - position 속성: 기본적인 위치 설정
  - z-index 속성: 요소간의 높이 차이 설정

CSS 포지셔닝 – Position (1/7)

- position: HTML 요소들의 위치 지정과 관련된 속성
  - Static: 기본 설정 (위치 요소가 지정되지 않은 상태)
  - Relative: 문서의 자연스러운 흐름 + 배치 기준점 변경
  - Absolute: 문서의 흐름에서 벗어남 + 절대 좌표 설정
  - Fixed: 문서의 흐름에서 벗어남 + 강제 위치 고정
  - Sticky: 문서의 자연스러운 흐름 + 강제 위치 고정

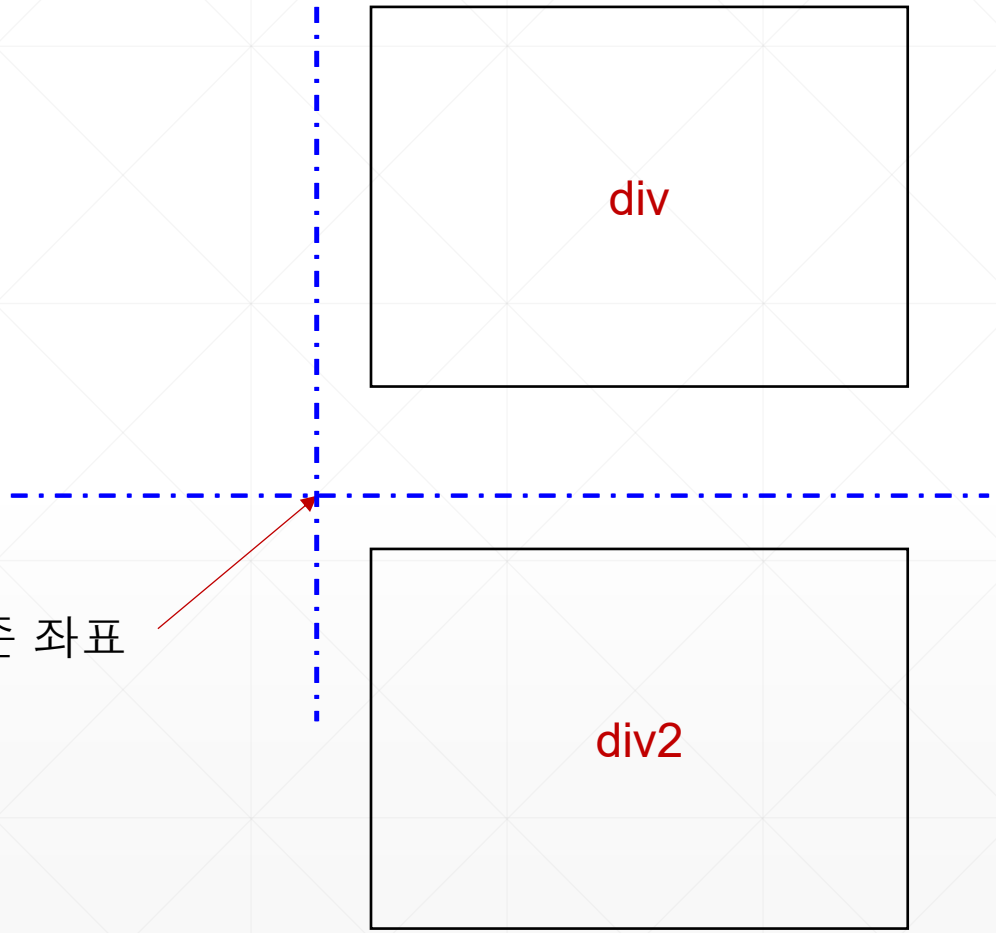
CSS 포지셔닝 – Position (2/7)

- Static: 문서의 일반적인 흐름을 따르는 기본 속성
  - 위치 지정과 관련된 기본 (default) 설정
  - position이 static으로 설정될 경우, 추후 다룰 top/bottom/right/left, z-index와 같은 위치 관련 속성들이 동작하지 않음

- Relative: 문서의 일반적인 흐름을 따르지만 배치 기준점을 변경하기 위한 속성
  - top/bottom/left/right 속성을 이용하여 기준점 조절 가능
  - 각 속성의 값은 기준점 조절을 위한 상대 위치로 활용됨
  - 양수의 경우 안쪽으로, 음수의 경우 바깥쪽으로 기준점이 이동됨
  - 기준점 조절 값이 설정되지 않을 경우 static과 동일하게 동작

CSS 포지셔닝 – Position (3/7)

- Relative 기준점 변경
  - 배치 기준 좌표 변경
  - 양수의 경우 안쪽으로 이동
  - 음수의 경우 바깥쪽으로 이동



일반적인 흐름에서 다음 div의 기준 좌표

CSS 포지셔닝 – Position (4/7)

Relative 실습)

- HTML 및 CSS 구성

```html
<div></div>
<div class="top"></div>
<div class="left"></div>
<div class="right"></div>
```

```css
div{
    position:relative;
    width:100px;
    height:100px;
    background-color: beige;
    border:black solid;
}
.top { top: 10px; }
.left { left: 10px; }
.right { right: 10px; }
```

Relative 설정 내역

- .top: 일반 흐름에서 상단 기준 (안쪽으로) 10px 이동하여 위치 조절
- .left: 일반 흐름에서 좌측 기준 (안쪽으로) 10px 이동하여 위치 조절
- .right: 일반 흐름에서 우측 기준 (안쪽으로) 10px 이동하여 위치 조절

CSS 포지셔닝 – Position (5/7)

- Absolute: 문서의 흐름과 상관없이 원하는 위치에 배치
  - 위치 속성이 설정된 부모 요소를 기준으로 위치 기준점을 변경
  - 위치 속성이 설정된 요소?
    - Position:static 으로 설정되어 있지 않은 요소!
  - 위치 속성이 설정된 부모 요소가 없을 경우
    - Body를 기준으로 조절
  - top/bottom/right/left 속성값을 이용하여 조절

CSS 포지셔닝 – Position (6/7)

- Absolute 실습)

```html
<div class="parent">
    <div class="top"></div>
    <div class="left"></div>
</div>
<div class="right"></div>
```

```css
div{
    position:absolute;
    width:100px;
    height:100px;
    background-color: beige;
    border:black solid;
}
.parent {width:300px;height:300px;margin:100px;}
.top{ top:100px; }
.left{ left:100px; }
.right{ right:100px; }
```

- 기준점
  - Top/left의 기준점: parent div
  - Right의 기준점: body

CSS 포지셔닝 – Position (7/7)

- Fixed: 문서의 흐름과 상관없이 원하는 위치에 배치
  - 부모 요소가 아닌 브라우저 창 기준
  - 브라우저 창 왼쪽 위 꼭지점(0,0) 기준으로 좌표 계산
  - 브라우저 창 **화면을 스크롤하더라도 계속 같은 위치에 고정**

```
.right {
  position:fixed;
  top:10px;
  right: 100px;
}
```

- Sticky
  - Relative와 fixed의 하이브리드 형태
  - top/bottom/right/left에 선언된 간격이 보장될 경우 relative 동작
  - top/bottom/right/left에 선언된 간격이 보장되지 않을 경우 해당 위치에서 fixed 동작

```
.parent {
  width: 300px;
  height: 300px;
  margin: 100px;
  position:sticky;
  top:10px;
}
```

CSS 포지셔닝 – Z-index (1/6)

- 요소가 쌓이는 순서를 설정할 수 있는 속성
  - 양수, 0, 음수 등의 값을 설정 가능
  - 큰 z-index 수를 가진 요소가 위에 쌓이게 됨
    - 동일한 z-index를 가질 경우, 뒤에 선언된 html 요소가 위에 쌓임
  - 위치 속성이 설정된 요소에만 동작함 (static은 불가)
  - Z-index는 같은 스택 부모(컨텍스트) 안의 자식들 끼리 경쟁한다!

CSS 포지셔닝 – Z-index (2/6)

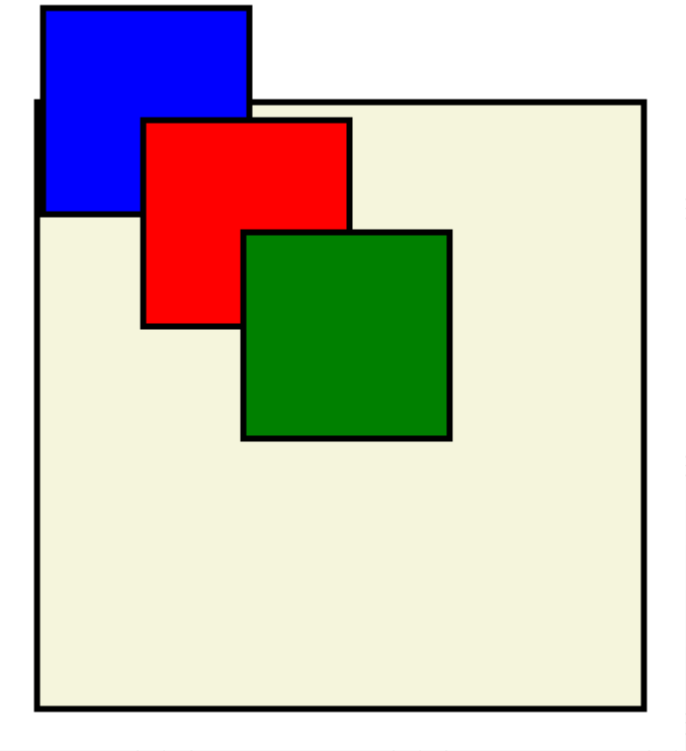
- Z-index 실습)

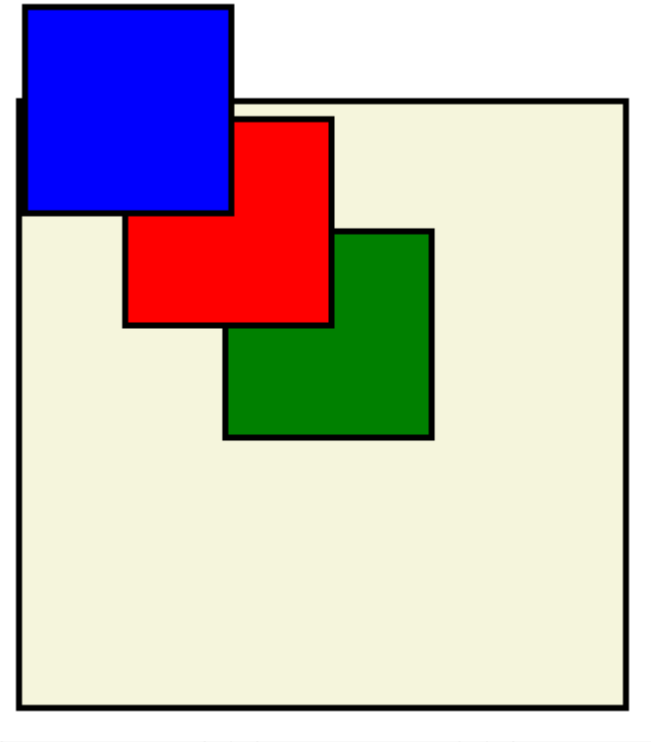
```
div{
    position:relative;
    width:100px;
    height:100px;
    background-color: beige;
    border:black solid;
}
.parent {width:300px;height:300px;margin:100px;}
.top{ top:-50px; background-color: blue; }
.left{ top:-100px;left:50px; background-color: red;}
.right{ top:-150px;left:100px; background-color: green;}
```

```
<div class="parent">
    <div class="top"></div>
    <div class="left"></div>
    <div class="right"></div>
</div>
```

CSS 포지셔닝 – Z-index (3/6)

- 실행 예)





➢ 두 번째 예제와 같이 배치시켜 보자!

CSS 포지셔닝 – Z-index (4/6)

- Z-index 변경 쌓기 순서 조절 가능

```
.top {
    top: -50px;
    background-color: blue;
    z-index: 10;
}

.left {
    top: -100px;
    left: 50px;
    background-color: red;
    z-index: 5;
}

.right {
    top: -150px;
    left: 100px;
    background-color: green;
    z-index: 1;
}
```

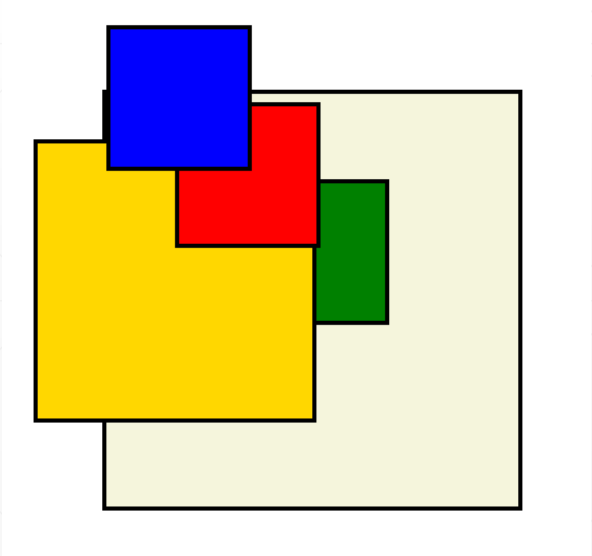
CSS 포지셔닝 – Z-index (5/6)

- 두 번째 실습) – 스태킹 컨텍스트

```
.uncle {
    top: -370px;
    left:50px;
    width:200px;
    height:200px;
    background-color:gold;
    z-index:1;
}
```

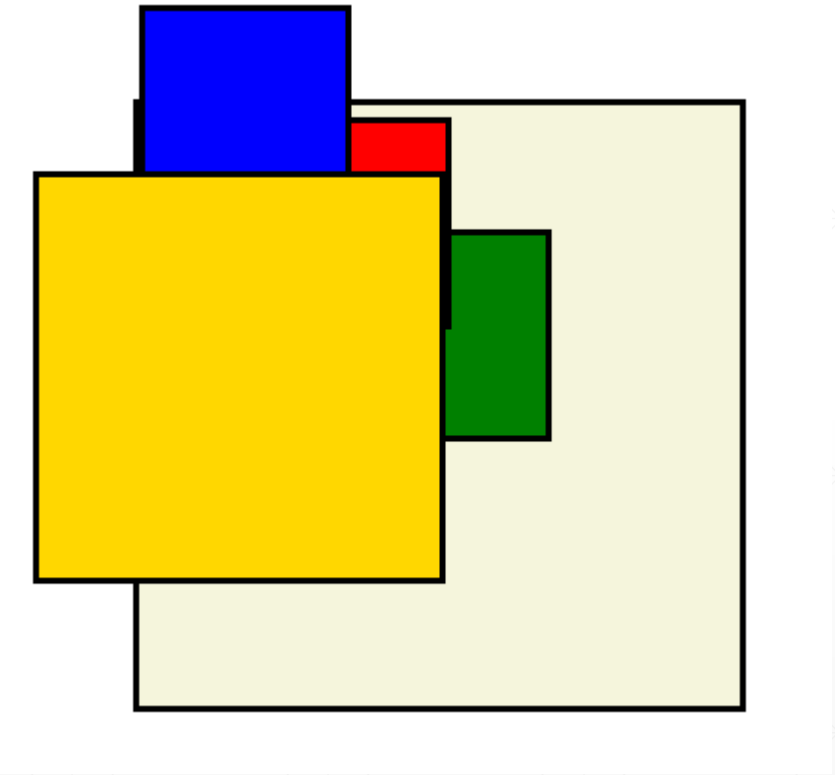
```
<div class="parent">
    <div class="top"></div>
    <div class="left"></div>
    <div class="right"></div>
</div>
<div class="uncle"></div>
```

- 실행 예)
  - Uncle division은 어디에 위치하고 있는가?
  - 왜 저 사이에 끼어 있을까? → 같은 스택 부모
  - Uncle의 z-index를 변경해보자!



CSS 포지셔닝 – Z-index (6/6)

- Parent div에 새로운 스택 컨텍스트 수립
  - Parent에 z-index: 0 추가



- 실행 예)
  - Uncle이 parent의 모든 블럭들보다 위에 있음
  - 왜?
    - Parent의 스택 컨텍스트 레벨: 0
    - Uncle의 스택 레벨: 1

- Parent의 z-index를 2로 변경하면 어떻게 될까?